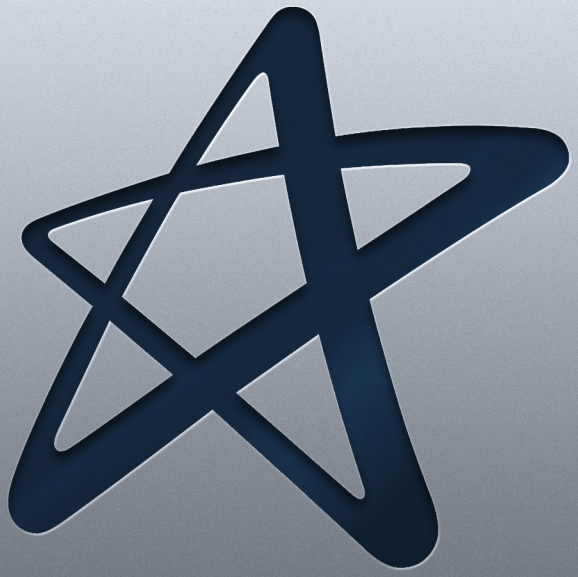
Ciência de Dados e Aprendizagem de Máquina



Cruzeiro do Sul Virtual
Educação a distância

Regras de Associação

Responsável pelo Conteúdo:
Prof. Dr. Luciano Rossi

Revisão Textual:
Esp. Laryssa Fazolo

UNIDADE Regras de Associação

- Conceito de Regras de Associação;
- O Algoritmo *Apriori*;
- O Algoritmo *FP-Growth*.



OBJETIVO DE APRENDIZADO

- Analisar as principais características da tarefa de descoberta de regras de associação, por meio da definição de alguns de seus principais algoritmos com a respectiva aplicação prática.

Orientações de estudo

Para que o conteúdo desta Disciplina seja bem aproveitado e haja maior aplicabilidade na sua formação acadêmica e atuação profissional, siga algumas recomendações básicas:



Assim:

- ✓ Organize seus estudos de maneira que passem a fazer parte da sua rotina. Por exemplo, você poderá determinar um dia e horário fixos como seu "momento do estudo";
- ✓ Procure se alimentar e se hidratar quando for estudar; lembre-se de que uma alimentação saudável pode proporcionar melhor aproveitamento do estudo;
- ✓ No material de cada Unidade, há leituras indicadas e, entre elas, artigos científicos, livros, vídeos e *sites* para aprofundar os conhecimentos adquiridos ao longo da Unidade. Além disso, você também encontrará sugestões de conteúdo extra no item **Material Complementar**, que ampliarão sua interpretação e auxiliarão no pleno entendimento dos temas abordados;
- ✓ Após o contato com o conteúdo proposto, participe dos debates mediados em fóruns de discussão, pois irão auxiliar a verificar o quanto você absorveu de conhecimento, além de propiciar o contato com seus colegas e tutores, o que se apresenta como rico espaço de troca de ideias e de aprendizagem.

Conceito de Regras de Associação

A descrição do processo de mineração de regras de associação deve ser precedida de uma descrição do conjunto de dados considerado para essa tarefa. Os conjuntos de dados descritos para as tarefas anteriores apresentavam uma característica comum: eram compostos por objetos representados por um conjunto de atributos.

Há outro tipo de conjunto de dados que relaciona itens que pertencem à mesma transação. Uma transação é um subconjunto de itens que são observados em um mesmo contexto. Um exemplo clássico desse tipo de dados são aqueles oriundos das compras em um supermercado. Cada cliente, ao encerrar suas compras e passar pelo caixa de pagamento, representa uma transação e os itens que o cliente adquiriu serão associados a essa transação. Em um único dia podemos ter diversas transações, cada uma contendo diferentes itens que foram adquiridos em conjunto.

Esse tipo de conjunto de dados é chamado de base de dados transacionais e representa as relações de itens que, por exemplo, foram consumidos por um mesmo cliente do supermercado. Esses dados são coletados diariamente pelo supermercado e armazenados em um banco de dados, configurando uma base de dados rica em informações a respeito do padrão de consumo dos clientes.



Conheça mais sobre os conjuntos de dados transacionais no artigo "Diferença Entre Transacional e não Transacional". Disponível em: https://bit.ly/3tgFf8d

Veja na Tabela 1 um exemplo de dados transacionais que descreve um conjunto de sete transações e, para cada transação, temos uma relação de itens que foram comprados por um mesmo cliente. Assim, podemos dizer que a associação entre os itens consumidos descreve um padrão de consumo de cada cliente e, a partir desse tipo de dado, podemos identificar um tipo de padrão baseado nas regras de associação dos itens consumidos.

Tabela 1 – Exemplo de conjunto de dados transacional

Identificador da transação	Itens
T_1	{pão, leite}
T^2	{pão, fralda, leite}
T_3	{pão}
T_4	{pão, leite, ovo}
T_5	{leite}
T_6	{pão, fralda}
T_7	{pão, fralda, leite, ovo}

Fonte: Adaptado de CASTRO *et al.*, 2016

Uma regra de associação pode ser definida como um padrão descritivo que representa a probabilidade de itens serem observados em uma mesma transação. Veja que podemos descrever um conjunto de dados transacionais de forma similar às descrições anteriores. Somente que os atributos pertencentes aos registros serão os itens que compõem a base

de dados. Desse modo, as regras de associação são similares às regras de classificação, exceto que, no primeiro caso, podemos predizer qualquer atributo e não somente a classe.

Podemos derivar uma grande quantidade de regras de associação a partir de pequenos conjuntos de dados transacionais. Assim, é comum que as regras consideradas sejam aquelas que atendam a dois conceitos: suporte e confiança.



Aprofunde seus conhecimentos a respeito de suporte, confiança e *lift*, lendo a dissertação de mestrado. Disponível em: https://bit.ly/3Jj0QCx

O suporte é referente ao número de transações que são corretamente preditas por uma determinada regra de associação, esse termo também é conhecido como cobertura da regra de associação. Nesse sentido, a confiança é a proporção entra as transações preditas corretamente e o total de transações nas quais a regra está inserida. Esse conceito também pode ser descrito como a acurácia ou a certeza de uma determinada regra.

As etapas previstas para o processo de obtenção de regras de associação envolvem um pré-processamento, a obtenção de um conjunto de itens frequentes, a própria mineração das regras e a avaliação dos resultados.

O pré-processamento segue de acordo com as descrições feitas para essa etapa em tarefas anteriores, nas quais observamos ações relativas à limpeza, integração, redução, transformação e discretização dos dados. Além disso, a tarefa de extração de regras de associação pode ter que ser feita sobre um conjunto de dados binários. Assim, a transformação do conjunto de dados original em um conjunto binário pode ser uma etapa adicional.

A definição de um limite inferior arbitrário, para o número de itens que farão parte da análise, tem por objetivo identificar um conjunto de itens mais frequentes. Essa etapa do processo de obtenção de regras de associação tem por finalidade a redução do volume de associações analisadas, visto que podemos ter muitas associações derivadas de pequenos conjuntos de dados, o que pode ser um problema para o processamento desses dados.

Considerando o exemplo da Tabela 1, poderíamos extrair um conjunto de regras de associação que teriam a seguinte forma:

$$\{fralda\} \rightarrow \{leite\}$$

$$\{pão, fralda\} \rightarrow \{leite, ovo\}$$

$$\{pão, leite\} \rightarrow \{ovo\}$$

As regras anteriores podem ser lidas como: há muitos clientes que compraram pão e ovo na mesma transação. Nesse caso, o símbolo de implicação ($\rightarrow$) nas regras não indica causa, e sim coocorrência.

No contexto da mineração de regras de associação, um conjunto de itens do domínio é conhecido como *itemset*. Nesse sentido, um *itemset* que é composto por *k* itens é denominado de *k-itemset*. Considerando nosso conjunto de dados exemplo, um *2-itemset* é

um conjunto com dois elementos, por exemplo, $\{pão, leite\}$. Da mesma forma, o *itemset* $\{pão, leite, ovo\}$ é um *3-itemset*.

Uma regra de associação tem o formato $A \rightarrow B$, que pode ser lido como se o *itemset* A estivesse presente em uma transação, então o *itemset* B, também, estará presente, considerando algum grau de confiança. Nesse contexto, dizemos que A é a premissa e B é a conclusão.

Suponha $I = \{pão, fralda, leite, ovo\}$ como o conjunto de todos os itens do domínio, assim, A e $B \subset I$ e $A \cap B = \varnothing$, ou seja, não existem itens comuns entre a premissa e a conclusão. Considere a seguinte regra de associação como exemplo ilustrativo: $\{fralda\} \rightarrow \{pão, leite\}$.

Para familiarizar-se com a notação anterior, referente à Teoria dos Conjuntos, leia o artigo. Disponível em: https://bit.ly/3KSQqK9

As regras de associação, em um conjunto de dados transacional, podem ser fortes ou fracas. A forma de determinar se uma regra é forte ou fraca considera duas medidas principais, denominadas suporte e confiança.

O suporte de um *itemset* referencia a frequência com que a regra aparece no conjunto de dados. Em outras palavras, a medida de suporte de uma regra de associação é a frequência com que os itens de um itemset ocorrem juntos nas transações.

Considerando nosso conjunto de dados exemplo, o *2-itemset* $\{pão, leite\}$ tem seus itens aparecendo, em conjunto, nas transações T_1, T_2, T_4 e T_7, assim, o suporte desse *itemset* é igual a $4/7$, pois, do total de sete transações, os itens do *itemset* em questão estão presentes em quatro dessas transações, o que corresponde a, aproximadamente, 57%.

Além da aplicação aos *itemsets*, a medida de suporte pode ser aplicada às regras. Nesse caso, a medida de suporte descreve a frequência com que os itens pertinentes aos *itemsets* de uma determinada regra aparecem juntos em transações. Em outras palavras, o suporte da regra $A \rightarrow B$ é o suporte do itemset $C = A \cup B$.

Considerando o conjunto de dados exemplo, que apresenta a regra $\{fralda\} \rightarrow \{pão, leite\}$, o suporte para essa regra será dado a partir do suporte do *itemset* $\{pão, fralda, leite\}$, que é a união dos itemsets que compõem a regra. Observe que os itens pertinentes ao referido *itemset* estão presentes, em conjunto, nas transações T_2 e T_7, assim, o suporte do *itemset* será $2/7 = 28\%$, valor que também corresponde ao suporte da regra descrita inicialmente.

Nesse contexto, o suporte aplicado a uma regra pode ser descrito da seguinte forma:

$$suporte_{regra}(A \rightarrow B) = \frac{contagem(A \cup B)}{contagem(T)}$$

em que $contagem(A \cup B)$ é o número de transações que contêm os itens do *itemset* $(A \cup B)$ e $contagem(T)$ é o número total de transações no conjunto de dados.

A segunda medida utilizada para definir se uma regra é forte ou fraca é denominada confiança. A medida de confiança considera somente as regras e não os *itemsets*. Como o próprio nome anuncia, a confiança traz a ideia de importância e confiabilidade de uma regra, em função de sua possibilidade de ocorrência.

Vamos considerar, novamente, a regra $\{fralda\} \rightarrow \{pão, leite\}$. Para calcular a confiança dessa regra, utilizamos a seguinte fórmula:

$$confiança_{regra}(A \rightarrow B) = \frac{suporte_{regra}(A \rightarrow B)}{suporte_{itemset}(A)}$$

Em outros termos, a confiança de uma regra é quociente entre o suporte da regra e o suporte da premissa da regra. Considerando o exemplo, temos que o suporte da regra é igual a 28% (calculado no exemplo anterior) e o suporte do *itemset* $\{fralda\}$ é igual a 43%, visto que esse item ocorre em três de sete regras. Assim, a confiança da regra será de 65% (28/43). A interpretação da confiança da regra, do nosso exemplo, é de que em 65% das vezes em que a premissa ocorre, a conclusão também ocorre.

Tanto o suporte quanto a confiança são medidas utilizadas como parâmetro de comparação com valores definidos arbitrariamente, de acordo com o objetivo pretendido. Dizemos que um item, ou *itemset*, é frequente se o seu suporte é maior que um limiar previamente definido.

A tarefa de encontrar itens frequentes é custosa, pois implica muitos itens que precisam ser testados. Quando um k-*itemset* é frequente, todos os $(k-1)$-*itemsets* derivados também serão. Por exemplo, suponha que um 10-*itemset* é frequente, visto que seu suporte é maior que um determinado limiar pré-estabelecido, nesse caso, todos os j-*itemsets,* sendo $j = 1,2,3,4,5,6,7,8,9$, também serão itemsets frequentes. Isso implicará em um total de $2^{10} - 1$ itemsets frequentes. Esse grande número de *itemsets* é devido a um princípio, denominado Propriedade *Apriori*, que diz: "todo subconjunto não vazio de um *itemset* frequente é também frequente".

Um conjunto de dados transacionais que contenha apenas quatro itens pode gerar um conjunto com 50 regras de associação, para as quais ainda teríamos de calcular o suporte e a confiança.

Uma forma de tornar a detecção de regras de associação um processo computacionalmente viável é a redução de algum elemento associado ao processo. Assim, poderíamos utilizar técnicas para a redução do número de candidatos, transações ou comparações.

O Algoritmo *Apriori*

Definições

Existem diferentes algoritmos que podem ser considerados para a extração de regras de associação. A maior parte desses algoritmos são derivações ou melhorias realizadas a partir do algoritmo Apriori.

O algoritmo Apriori considera duas subtarefas principais. Na primeira, são identificados todos os conjuntos de itens frequentes, de acordo com um limite previamente definido. A segunda subtarefa consiste na geração das regras desejadas por meio da identificação dos itens candidatos, que vão sendo eliminados à medida que não atendam à frequência estabelecida. Ao final do processo, tem-se um conjunto de itens que atendam à frequência estabelecida.

A primeira fase do algoritmo Apriori é apresentada no Algoritmo 1.

Algoritmo 1, Primeira fase do algoritmo Apriori

Entrada

- T : um conjunto de transações;
- s : um suporte mínimo;
- $k = 1$: um contador;

Saída

- L : uma lista de itemsets frequentes;

Etapas

1. Calcular o suporte de cada $1-$ itemset de T ;
2. Selecionar os $1-$ itemsets com suporte $\geq s$ e inserir em L_k ;
3. Enquanto $L_k \neq \varnothing$ faça:
   - » Combinar os itemsets de L_k gerando o conjunto $(k+1)-$ itemsets candidatos;
   - » Calcular o suporte de cada novo itemset;
   - » $k = k + 1$;
   - » Selecionar os itemsets com suporte $\geq s$ e inserir em L_k ;
   - » $L \cup L_k$.

A primeira iteração do algoritmo Apriori realiza o cálculo do suporte dos $1-$ *itemsets* e seleciona aqueles que apresentam o valor dessa medida maior ou igual ao valor limiar. Nesse exemplo, consideraremos como limiar $s = 40\%$. O resultado dessa iteração é descrito na Tabela 2.

Tabela 2 – Cálculo do suporte para os 1-*itemsets*. Resultado da primeira fase do algoritmo *Apriori*

1-*itemset* (candidatos)	Subconjunto de transações nas quais o *itemset* ocorre	Cálculo do suporte *Contagem (T)* = 7	Suporte (%)
$\{pão\}$	$\{T_1, T_2, T_3, T_4, T_6, T_7\}$	$s_{pão} = \frac{6}{7} = 0,86$	86%
$\{fralda\}$	$\{T_2, T_6, T_7\}$	$s_{fralda} = \frac{3}{7} = 0,43$	43%
$\{leite\}$	$\{T_1, T_2, T_4, T_5, T_7\}$	$s_{leite} = \frac{5}{7} = 0,71$	71%
$\{ovo\}$	$\{T_4, T_7\}$	$s_{ovo} = \frac{2}{7} = 0,29$	29%

Fonte: Adaptada de DA SILVA, 2017

Analisando os resultados apresentados na Tabela 2, é possível notar que os 1-*itemsets* $\{pão\}$, $\{fralda\}$ e $\{leite\}$ têm um suporte maior que o limiar estabelecido (40%), assim, esses 1-*itemsets* serão utilizados na iteração posterior do algoritmo. Por outro lado, o 1-*itemset* $\{ovo\}$ tem suporte igual a 28%, valor que é inferior ao limiar (40%), portanto esse 1-*itemset* não segue para a próxima iteração do algoritmo.

A segunda iteração do algoritmo Apriori realiza o cálculo do suporte dos 2-*itemsets*. Veja que a iteração anterior resultou em três 1-*itemsets* ($\{pão\}, \{fralda\}, \{leite\}$) e, se combinarmos esses itens, dois a dois, teremos os seguintes 2-*itemsets* possíveis: $\{pão, fralda\}$, $\{pão, leite\}$ e $\{fralda, leite\}$. Esses 2-*itemsets* serão considerados pela segunda iteração do algoritmo, cujo resultado é apresentado na Tabela 3.

Tabela 3 – Cálculo do suporte para os 2-*itemsets*. Resultado da primeira fase do algoritmo *Apriori*

2-*itemset* (candidatos)	Subconjunto de transações nas quais o *itemset* ocorre	Cálculo do suporte *Contagem (T)* = 7	Suporte (%)
$\{pão, fralda\}$	$\{T_2, T_6, T_7\}$	$s_{\{pão, fralda\}} = \frac{3}{7} = 0,43$	43%
$\{pão, leite\}$	$\{T_1, T_2, T_4, T_7\}$	$s_{\{pão, leite\}} = \frac{4}{7} = 0,57$	57%
$\{fralda, leite\}$	$\{T_2, T_7\}$	$s_{\{fralda, leite\}} = \frac{2}{7} = 0,29$	29%

Fonte: Adaptada de DA SILVA, 2017

Veja que o 2-*itemset* $\{fralda, leite\}$ tem suporte igual a 29%, valor que é inferior ao limiar estabelecido para essa medida (40%), assim, esse 2-*itemset* não seguirá para a próxima iteração do algoritmo *Apriori*. Os outros dois 2-*itemsets*, $\{pão, fralda\}$ e $\{pão, leite\}$, têm valores de suporte iguais a 43% e 57%, respectivamente, assim, esses 2-*itemset* serão considerados para a formação de subconjuntos 3-*itemsets*, que seguem

para a iteração posterior. A partir dos itens dos 2-*itemsets* selecionados, é possível obter um único 3-*itemset*: $\{pão, fralda, leite\}$. O valor do suporte desse 3-*itemset* é apresentado na Tabela 4.

Tabela 4 – Cálculo do suporte para o 3-*itemset*. Resultado da Fase 1 do algoritmo *Apriori*

2-*itemset* (candidatos)	Subconjunto de transações nas quais o *itemset* ocorre	Cálculo do suporte *Contagem (T)* = 7	Suporte (%)
$\{pão, fralda, leite\}$	$\{T_2, T_7\}$	$s_{\{pão, fralda, leite\}} = \frac{2}{7} = 0,28$	28%

Fonte: Adaptada de DA SILVA, 2017

O cálculo do suporte do 3-*itemset*, apresentado na Tabela 4, resultou em um valor igual a 28%, o qual é inferior ao limiar de 40% estabelecido inicialmente. Assim, esse 3-*itemset* é descartado.

A finalização da execução do algoritmo *Apriori* ocorre quando não é possível formar novos *k-itemsets*. Veja que para 4-*itemset* não é possível a formação, pois contamos com somente três itens. Assim, o resultado do algoritmo *Apriori* é a lista $L = \{\{i_1, i_2, \}, \{i_1, i_3\}\}$.

A segunda fase do algoritmo *Apriori* realizará a geração das regras a partir dos *itemsets* da lista L. Segundo DA SILVA *et al.* (2017), a segunda fase do algoritmo *Apriori* apresenta os seguintes passos:

Algoritmo 2, Segunda fase do algoritmo *Apriori*

Entrada

- *L*: uma lista de *itemsets* frequentes;
- *c:* uma confiança mínima;

Saída

- *C:* um conjunto de regras de associação fortes;

Etapas

1. Para cada itemset frequente em L faça:
   - » Gerar todas as regras possíveis (A⟶B) combinando os conjuntos não vazios, lembrando que A∩B=∅;
2. Para cada regra gerada faça:
   - » Calcule a confiança da regra;
   - » Se a confiança da regra ≥c inserir a regra no conjunto C.

A partir do 2-*itemset* $\{pão, fralda\}$, da lista L, é possível gerar as seguintes regras: $\{pão \rightarrow fralda\}$e $\{fralda \rightarrow pão\}$, do mesmo modo, o 2-*itemset* $\{pão \rightarrow leite\}$, da lista L, gerará as regras:$\{pão \rightarrow leite\}$ e $\{leite \rightarrow pão\}$. Após a geração das regras, a segunda fase do algoritmo Apriori realizará o cálculo da confiança de cada regra e, considerando o limiar estabelecido, decidirá se a regra será considerada. Veja, na Tabela 5, os resultados da segunda fase do algoritmo.

Tabela 5 – Cálculo da confiança das regras obtidas a partir da segunda fase do algoritmo *Apriori*

A	B	Suporte/Regra	Suporte/Promissa	Confiança/Regra	Inserção em C
Pão	Fralda	0,43	0,86	0,50	Não
Fralda	Pão	0,43	0,43	1,00	Sim
Pão	Leite	0,57	0,86	0,66	Não
Leite	Pão	0,57	0,71	0,80	Sim

Fonte: Adaptada de DA SILVA, 2017

Veja que no universo de sete transações do conjunto de dados-exemplo existem, apenas, duas regras consideradas fortes: $fralda \rightarrow pão$ e $leite \rightarrow pão$, cujos valores de confiança são 1,00 e 0,80, respectivamente. As outras duas regras não foram consideradas, pois seus valores de confiança ficaram abaixo do limiar estabelecido de 70%, portanto são regras fracas.

A regra $fralda \rightarrow pão$, cujo suporte é de 43% e a confiança é de 100%, indica que a frequência de observação dessa regra no conjunto de dados é dada pelo suporte e que, quando o item $fralda$ aparece em uma transação, o item $pão$ sempre aparecerá também. A segunda regra $leite \rightarrow pão$, cujo suporte é de 57% e a confiança é de 80%, é mais frequente que a regra anterior, evidenciado pelo suporte, e $pão$ aparece em 80% das transações em que $leite$ aparece.

Aplicações com a Linguagem *R*

Considere o seguinte conjunto de dados para a ilustração do funcionamento do algoritmo *Apriori* a partir da linguagem de programação *R*:

```
Feijao,Salada,Parmegiana,Fritas,Arroz

NAO,SIM,NAO,NAO,SIM

NAO,NAO,NAO,SIM,NAO

NAO,NAO,SIM,SIM,NAO

NAO,NAO,SIM,NAO,SIM

NAO,NAO,SIM,SIM,NAO

SIM,NAO,SIM,SIM,SIM

NAO,NAO,NAO,SIM,NAO

SIM,NAO,SIM,SIM,SIM

NAO,NAO,NAO,NAO,SIM

NAO,NAO,SIM,SIM,NAO
```

O primeiro passo consiste na conversão do nosso conjunto de dados para um formato transacional. Para isso, utilizaremos o pacote **`arules`**, que pode ser instalado a partir do seguinte comando:

```
> install.packages("arules")

> library(arules)
```

Seja o conjunto de dados um objeto denominado dados, a conversão pode ser feita a partir dos comandos:

```
> bd <- as(dados, "transactions")

> inspect(bd)
```

Os quais resultarão no seguinte:

```
    items                                                    transactionID
[1] {Feijao=NAO,Salada=SIM,Parmegiana=NAO,Fritas=NAO,Arroz=SIM} 1
[2] {Feijao=NAO,Salada=NAO,Parmegiana=NAO,Fritas=SIM,Arroz=NAO} 2
[3] {Feijao=NAO,Salada=NAO,Parmegiana=SIM,Fritas=SIM,Arroz=NAO} 3
[4] {Feijao=NAO,Salada=NAO,Parmegiana=SIM,Fritas=NAO,Arroz=SIM} 4
[5] {Feijao=NAO,Salada=NAO,Parmegiana=SIM,Fritas=SIM,Arroz=NAO} 5
[6] {Feijao=SIM,Salada=NAO,Parmegiana=SIM,Fritas=SIM,Arroz=SIM} 6
[7] {Feijao=NAO,Salada=NAO,Parmegiana=NAO,Fritas=SIM,Arroz=NAO} 7
[8] {Feijao=SIM,Salada=NAO,Parmegiana=SIM,Fritas=SIM,Arroz=SIM} 8
[9] {Feijao=NAO,Salada=NAO,Parmegiana=NAO,Fritas=NAO,Arroz=SIM} 9
[10]{Feijao=NAO,Salada=NAO,Parmegiana=SIM,Fritas=SIM,Arroz=NAO} 10
```

A linguagem de programação *R* disponibiliza uma série de funções que implementam diferentes tarefas de aprendizado de máquina. Para a extração de regras de associação, especificamente para o algoritmo *Apriori*, utilizaremos a função **apriori**. A base de dados, escolhida para esse exemplo, apresenta valores categóricos (SIM, NÃO). Nesse caso, a função **apriori** não é capaz de decidir, de forma automática, a diferença entre esses valores. Assim, precisamos "dizer" para a função qual é o valor que representa a nulidade para os atributos, ou seja, qual é o valor que descreve que aquele item, ou atributo, não pertence àquela transação. Essa questão é resolvida a partir do parâmetro itens na transação. A sintaxe completa para a utilização da função é descrita a seguir:

```
regras <- apriori(base de dados, parâmetros, itens na transação)
```

Veja que, na representação anterior, o objeto **regras** é aquele que armazenará as regras de associação resultante da aplicação do algoritmo *Apriori*. Nesse contexto, **apriori** é a função que implementa o algoritmo, **base de dados** é o conjunto de dados transacional, **parâmetros** descreve, entre outras possibilidades, as medidas de suporte e confiança e **itens na transação** descreve os itens e seus respectivos valores nulos.

Considerando o nosso exemplo, a utilização da função é feita da seguinte forma:

```
Regras <- apriori (
```

```
   data = bd,
   parameter = list (
                  minlen=2,
                  supp=0.4,
                  conf=0.8
                  ),
appearance = list (
                none=c (
                      “Feijao=NAO”,
                      “Salada=NAO”,
                      “Parmegiana=NAO”,
                      “Arroz=NAO”
                      )
                )
             )
```

Na representação anterior, são considerados os valores de 40% e 80% para suporte e confiança, respectivamente. Esses valores são representados pelos parâmetros **supp** e **conf**. Para visualizar o resultado da mineração de regras de associação, podemos utilizar o comando **inspect(regras)** que resultará em:

```
    lhs                rhs          support confidence coverage lift     count

[1]{Parmegiana=SIM} => {Fritas=SIM} 0.5     0.8333333  0.6      1.190476 5
```

O resultado a aplicação anterior é a descrição da regra com os maiores valores de suporte e confiança. Nessa aplicação, considerou-se somente a presença de itens nas transações. Poderíamos estar interessados em analisar um outro aspecto, no qual o objetivo é as relações entre produtos não vendidos. Assim, poderíamos utilizar a mesma notação sem o parâmetro **appearance**. Assim, o resultado seria uma relação de regras nas quais teríamos, também, relações de itens ausentes nas transações. Veja a seguir o resultado deste exemplo:

```
     lhs                                 rhs          support confidence coverage lift      count
[1]  {Parmegiana=NAO}                 => {Feijao=NAO} 0.4     1.0000000  0.4      1.2500000 4
[2]  {Arroz=SIM}                      => {Salada=NAO} 0.4     0.8000000  0.5      0.8888889 4
[3]  {Arroz=NAO}                      => {Fritas=SIM} 0.5     1.0000000  0.5      1.4285714 5
[4]  {Arroz=NAO}                      => {Feijao=NAO} 0.5     1.0000000  0.5      1.2500000 5
[5]  {Arroz=NAO}                      => {Salada=NAO} 0.5     1.0000000  0.5      1.1111111 5
[6]  {Parmegiana=SIM}                 => {Fritas=SIM} 0.5     0.8333333  0.6      1.1904762 5
[7]  {Parmegiana=SIM}                 => {Salada=NAO} 0.6     1.0000000  0.6      1.1111111 6
[8]  {Fritas=SIM}                     => {Salada=NAO} 0.7     1.0000000  0.7      1.1111111 7
[9]  {Feijao=NAO}                     => {Salada=NAO} 0.7     0.8750000  0.8      0.9722222 7
[10] {Fritas=SIM,Arroz=NAO}           => {Feijao=NAO} 0.5     1.0000000  0.5      1.2500000 5
[11] {Feijao=NAO,Arroz=NAO}           => {Fritas=SIM} 0.5     1.0000000  0.5      1.4285714 5
[12] {Feijao=NAO,Fritas=SIM}          => {Arroz=NAO}  0.5     1.0000000  0.5      2.0000000 5
[13] {Fritas=SIM,Arroz=NAO}           => {Salada=NAO} 0.5     1.0000000  0.5      1.1111111 5
[14] {Salada=NAO,Arroz=NAO}           => {Fritas=SIM} 0.5     1.0000000  0.5      1.4285714 5
[15] {Feijao=NAO,Arroz=NAO}           => {Salada=NAO} 0.5     1.0000000  0.5      1.1111111 5
[16] {Salada=NAO,Arroz=NAO}           => {Feijao=NAO} 0.5     1.0000000  0.5      1.2500000 5
[17] {Parmegiana=SIM,Fritas=SIM}      => {Salada=NAO} 0.5     1.0000000  0.5      1.1111111 5
[18] {Salada=NAO,Parmegiana=SIM}      => {Fritas=SIM} 0.5     0.8333333  0.6      1.1904762 5
[19] {Feijao=NAO,Parmegiana=SIM}      => {Salada=NAO} 0.4     1.0000000  0.4      1.1111111 4
[20] {Feijao=NAO,Fritas=SIM}          => {Salada=NAO} 0.5     1.0000000  0.5      1.1111111 5
[21] {Feijao=NAO,Fritas=SIM,Arroz=NAO} => {Salada=NAO} 0.5    1.0000000  0.5      1.1111111 5
[22] {Salada=NAO,Fritas=SIM,Arroz=NAO} => {Feijao=NAO} 0.5    1.0000000  0.5      1.2500000 5
[23] {Feijao=NAO,Salada=NAO,Arroz=NAO} => {Fritas=SIM} 0.5    1.0000000  0.5      1.4285714 5
[24] {Feijao=NAO,Salada=NAO,Fritas=SIM} => {Arroz=NAO}  0.5    1.0000000  0.5      2.0000000 5
```

Figura 1
Fonte: Reprodução

Veja que a regra [1] indica que, em todas as transações nas quais não havia o item Parmegiana, também não havia o item Feijao, e a ausência desses itens, em conjunto, foi observada em 40% das transações.

O Algoritmo *FP-Growth*

Definições

O algoritmo *FP-Growth* (*frequent-pattern growth*) difere do algoritmo Apriori por considerar uma estratégia mais eficiente. Essa estratégia possibilita que os itemsets frequentes sejam identificados sem a criação de listas de candidatos. O processo considera a verificação da base de dados uma única vez para a identificação da frequência de cada 1-*itemset*.

Os 1-*itemsets* mais frequentes são ordenados em uma lista de acordo com suas respectivas frequências. Considerando somente uma nova verificação na base de dados, os *itemsets* são organizados em uma estrutura hierárquica. As etapas do algoritmo *FP-Growth* são descritas a seguir.

Algoritmo 3, FP-Growth

Entrada

- TID: um conjunto de transações;
- $f_{\min}$: suporte mínimo.

Saída

- L : uma lista de itemsets frequentes.

Etapas

1. Calcular a frequência de cada 1-itemset $\in TID$;
2. Selecionar os 1-itemsets com frequência $\geq f_{\min}$;
3. Ordenar os itemsets selecionados em uma lista L_{ord};
4. Inicializar uma árvore com raiz A;
5. Para cada transação $t \in TID$ faça:
   - » $raiz\ atual = A$;
   - » Para cada $i \in L_{ord}$ presente em t faça:
   - » Se i é nó filho da $raiz\ atual$:
   - » Então:
   - » Visitar o nó filho;
   - » Incrementar o contador do rótulo do nó filho;
   - » $raiz\ atual = nó\ filho$;
   - » Senão:
   - » Criar um ramo na árvore para i;
   - » Inicializar o contador de i com 1;
   - » $raiz\ atual = nó\ do\ novo\ ramo$;
6. Para cada $i \in L_{ord}$, em ordem inversa faça:
   - » Encontrar os nós de i e para cada nó j faça:
   - » Extrair os ramos que precedem j;
   - » Associar aos nós do ramo a frequência de j;
   - » Combinar os ramos associados somando as frequências dos nós;
   - » Excluir os nós com frequência $< f_{min}$;
   - » Combinar i com os nós da subárvore associada;
   - » Inserir os itemsets em L.

Fonte: Adaptado de DA SILVA *et al.*, 2017

Aplicações com a Linguagem *R*

A base de dados Iris é um conjunto de dados multivariados proposto por Ronald Fisher, biólogo britânico, em 1936. Esse conjunto de dados consiste em 50 amostras de cada uma das três espécies de flor (*Iris setosa*, *Iris virginica* e *Iris versicolor*). Os atributos de cada amostra são o comprimento e a largura das sépalas e pétalas. Com base na combinação desses atributos, Fisher desenvolveu um modelo discriminante linear capaz de distinguir as espécies entre si. Esse conjunto de dados é possível de ser carregado no *RStudio* a partir do seguinte comando:

```
data("iris").
```

Para mais informações sobre a base de dados Iris, acesse o artigo "Dados de demonstração do Iris para tutoriais de Python e R com machine learning do SQL". Disponível em: https://bit.ly/364KNK9

Podemos verificar o conjunto de dados Iris, após seu carregamento, a partir de seu nome (**iris**), o que resultará na seguinte listagem:

```
      Sepal.Length   Sepal.Width   Petal.Length'   Petal.Width   Species
1          5.1           3.5           1.4           0.2        setosa
2          4.9           3.0           1.4           0.2        setosa
3          4.7           3.2           1.3           0.2        setosa
4          4.6           3.1           1.5           0.2        setosa
5          5.0           3.6           1.4           0.2        setosa
...
```

A função que implementa o algoritmo *FP-Growth* na linguagem *R* é denominada **fpgrowth**, disponível no pacote **rCBA**. Antes da utilização do conjunto de dados, é preciso uma preparação, transformando os valores em fatores, a partir do seguinte comando:

```
train <- sapply(iris,as.factor)
```

Como resultado, teremos o seguinte:

```
     Sepal.Length Sepal.Width Petal.Length Petal.Width Species
[1,] "5.1"        "3.5"       "1.4"        "0.2"       "setosa"
[2,] "4.9"        "3"         "1.4"        "0.2"       "setosa"
[3,] "4.7"        "3.2"       "1.3"        "0.2"       "setosa"
[4,] "4.6"        "3.1"       "1.5"        "0.2"       "setosa"
[5,] "5"          "3.6"       "1.4"        "0.2"       "setosa"
...
```

O próximo passo é a transformação para o formato transacional. Isso é feito considerando os seguintes comandos:

```
train <- data.frame(train, check.names=FALSE)

txns <- as(train,"transactions")
```

que produzirão o seguinte resultado:

```
items  transactionID

[1]{Sepal.Length=5.1,Sepal.Width=3.5,Petal.Length=1.4,Petal.
Width=0.2,Species=setosa}     1

[2]{Sepal.Length=4.9,Sepal.Width=3,Petal.Length=1.4,Petal.
Width=0.2,Species=setosa}     2
```

```
[3]{Sepal.Length=4.7,Sepal.Width=3.2,Petal.Length=1.3,Petal.
Width=0.2,Species=setosa}     3

[4]{Sepal.Length=4.6,Sepal.Width=3.1,Petal.Length=1.5,Petal.
Width=0.2,Species=setosa}     4

[5]{Sepal.Length=5,Sepal.Width=3.6,Petal.Length=1.4,Petal.
Width=0.2,Species=setosa}     5

...
```

A utilização da função *FP-Growth* para a mineração de regras de associação é feita de acordo com a seguinte descrição:

```
rules = rCBA::fpgrowth(txns, support=0.03, confidence=0.03, maxLength=2,
consequent="Species", parallel=FALSE),
```

na qual:

- ***train***: quadro de dados ou transações de arules com dados de entrada;
- ***support***: suporte mínimo;
- ***confidence***: confiança mínima;
- ***maxLength***: máximo comprimento;
- ***consequent***: nome da coluna que é a classe-alvo;
- ***parallel***: indicador paralelo.

O resultado do comando anterior é um conjunto de 47 regras de associação, representadas da seguinte forma:

```
    lhs                    rhs                    support    confidence lift

[1] {Sepal.Length=4.8} => {Species=setosa}     0.03333333 1.0000000  3.0000000

[2] {Petal.Length=4}   => {Species=versicolor} 0.03333333 1.0000000  3.0000000

[3] {Petal.Width=0.1}  => {Species=setosa}     0.03333333 1.0000000  3.0000000

[4] {Petal.Width=1.9}  => {Species=virginica}  0.03333333 1.0000000  3.0000000

[5] {Petal.Width=1.2}  => {Species=versicolor} 0.03333333 1.0000000  3.0000000

...
```

Note que as regras consideram somente 1-*itemset* tanto para a premissa quanto para a conclusão, de acordo com o que foi parametrizado na função.

Podemos utilizar as regras de associação obtidas para predizer as categorias de cada uma das flores no conjunto de dados. Para isso, utilizamos o seguinte comando:

```
predictions <- rCBA::classification(train,rules)
```

em que o conjunto de treinamento será classificado de acordo com as regras obtidas. Nesse caso, a taxa de acerto do classificador fica em torno de 95%, esse valor pode ser verificado por meio do seguinte comando:

```
sum(as.character(train$Species)==as.character(predictions),na.rm=TRUE)/
length(predictions)
```

Em Síntese

Vimos que as práticas atuais produzem uma grande quantidade de dados, estima-se que a geração de dados em um único dia é maior que a observada desde a pré-história.

Essa produção de dados é resultado da crescente capacidade de processamento, das novas formas de interação na Internet e de inovações como a Internet das Coisas (IoT). Nesse contexto, somente cerca de 0,45% dos dados produzidos são explorados, o restante dos dados é o que chamamos de *Dark Data*. Assim, existe um grande campo de atuação para profissionais na Ciência de Dados.

A Ciência de Dados não é uma área de atuação nova, existiram outros termos para designar esse tipo de atuação, como: *Decision Support Systems* (*DDS*) nos anos 1970, *Executive Information Systems* (*EIS*) nos anos 1980, *Online Analytical Processing* (*OLAP*) nos anos 1990, *Business Inteligence* (*BI*) nos anos 2000 e *Data Science* nos anos 2010.

As tarefas de Mineração de Dados (*Data Mining*), as quais foram exploradas nesta disciplina, diferem em objetivo quando as comparamos com a área de Banco de Dados. Em Banco de Dados, podemos querer identificar os dez clientes com mais reclamações, enquanto em Data Mining, o objetivo pode ser encontrar os atributos que levam os clientes a reclamar.

Embora similares e possíveis de serem confundidas, a área de Banco de Dados considera um padrão para obter os dados que se adequem a ele, por outro lado, em *Data Mining*, os dados são considerados para se extrair um padrão pretendido.

As tarefas que foram exploradas nesta disciplina são importantes e o seu entendimento e aplicabilidade pode contribuir para o desenvolvimento profissional daqueles que pretendem atuar na área. Nesse contexto, seguem as tarefas que exploramos nesta disciplina, bem como um resumo de suas características, de acordo com Cavique (2014):

- **Classificação:** a partir de um conjunto de dados e classes bem definidas, às quais os dados possam pertencer, encontrar as características que definem cada classe;
- **Agrupamento:** a partir de conjunto de dados, definir X grupos que particionem esses dados de acordo com suas características;
- **Regras de associação:** a partir de um conjunto de dados, encontrar padrões que levem a um conjunto de regras $y \rightarrow z$, com $x = (y, z)$;
- **Regressão:** modela a dependência entre um conjunto de variáveis descritivas com uma variável dependente, ou seja, $x = (y,z)$ com , deseja-se encontrar $f(y) = z$;
- **Detecção de anomalias:** a partir de um conjunto de dados, encontrar um modelo que descreva esses dados e encontrar objetos anômalos, ou seja, fora do modelo proposto.

Material Complementar

Indicações para saber mais sobre os assuntos abordados nesta Unidade:

Vídeos

WEKA – Regras de Associação com Algoritmo *Apriori* (1ª parte)
https://youtu.be/DNI3aIJjSN8

WEKA – Regras de Associação com Algoritmo *Apriori* (2ª parte)
https://youtu.be/SEaX3UTnFog

Como se Tornar um Cientista de Dados de Sucesso
https://youtu.be/-vawLSNdo-A

Como Criar um Plano de Estudos para se Tornar um Cientista de Dados?
https://youtu.be/o8NpsLSkKUo

Referências

AGRAWAL, R.; SRIKANT, R. *Fast algorithms for mining association rules in large databases. In*: BOCCA, J. B.; JARKE, M.; ZANIOLO, C. (ed.). **VLDB 94:** *Proceedings of the 20th International Conference on Very Large Data Bases.* San Francisco: Morgan Kaufmann Publishers Inc., 1994. p. 487-499. Disponível em: <http://www.vldb.org/conf/1994/P487.PDF>.

CARVALHO, A. C. P. L. F. de. **Aprendizado de máquina**: árvores de decisão. São Paulo: ICMC/USP, 2017. (Notas de aula). Disponível em: <https://edisciplinas.usp.br/pluginfile.php/4672393/mod_resource/content/1/Aula-05.pdf>.

CASTRO, L. N.; FERRARI, D. G. **Introdução à mineração de dados:** conceitos básicos, algoritmos e aplicações. São Paulo: Saraiva, 2016.

CAVIQUE, L. ***Big data e data science***. Boletim da APDIO, n. 51, p. 11-14, 2014. Disponível em: <http://apdio.pt/documents/10180/16684/Boletim_51.pdf>.

FARIA, F. A. **Aprendizagem não supervisionada – agrupamento** (*Clustering*). São Paulo: UNIFESP, 2015. (Notas de aula). Disponível em: <https://www.ic.unicamp.br/~ffaria/ia1s2017/class14/class14-Agrupamento.pdf>.

FRANÇA, F. O. **Mineração de sentimentos**. São Paulo: UFABC, 2014. (Notas de aula).

GRUS, J. ***Data Science do zero:*** primeiras regras com o Python. Rio de Janeiro: Alta Books, 2019.

HAN, J.; PEI, J.; YIN, Y. ***Mining frequent patterns without candidate generation***. ACM SIGMOD *Record*, v. 29, n. 2, p. 1-12, 2000.

HAND, D. J.; MANNILA, H.; SMYTH, P. ***Principles of data mining (adaptive computation and machine learning)***. Cambridge: MIT Press, 2001.

ROSSI, L. **Técnicas para converter dados em conhecimento**. São Paulo: Universidade Cruzeiro do Sul, 2021.

SILVA, L. A. da; PERES, S. M.; BOSCARIOLI, C. **Introdução à mineração de dados**: com aplicações em R. Rio de Janeiro: Elsevier Brasil, 2017.

ZAKI, M. J. *et al.* ***Parallel algorithms for discovery of association rules***. *Data mining and knowledge discovery*, [s. l.], v. 1, n. 4, p. 343-373, 1997.


Cruzeiro do Sul
Educacional